HyperPage: Learned Page Layouts for Disk-Resident Graph Vector Indexes

Xinyu Zhang[1], Jingze Li[1], Baihua Zheng[2], Weiwei Sun[1]
[1]Fudan University, Shanghai, China [2]Singapore Management University, Singapore
{zhangxinyu, jingzeli20, wwsun}@fudan.edu.cn, bhzheng@smu.edu.sg

arXiv:2611.16149v1 [cs.DB] 12 Nov 2026

***Abstract*—Disk-resident graph indexes such as DiskANN store each vector with its graph edges as one record on SSD pages. A search needs the page of every record it expands, so its page reads depend on the page layout, that is, on which records share a page. Search agents trained with reinforcement learning query one such index run after run, and every graph traversal can be logged with the records it expands. We therefore use the logs of a finished run to lay out the index for a declared memory budget, so that later runs read fewer pages. In contrast, existing layouts such as Starling and MARGO are computed from the graph and the vectors and use no logs. Learning a layout from logs is challenging, because the logs must stay valid under every layout, the saving depends on what the budget caches, and the best layout is NP-hard to find. Our key observation is that DiskANN-style search routes on in-memory codes and on the records it expands, never on their page-mates. As a result, the logged traversals price every layout by replay, and cached records add no page reads. Accordingly, we propose HyperPage, which first builds layouts for the declared budget on the logged traversals of the uncached records. It then selects one layout, together with the split of the budget between hot records and a page cache, by replaying validation steps. In addition, a Lagrangian lower bound shows how far a layout may be from the optimum. We learn from one Search-R1 agent run and test on a later run with a different reward, over 1M and 21M passages. Across seven cache budgets, HyperPage reads 5.2% to 21.2% fewer pages than MARGO, and no compared layout reads fewer. The advantage is largest when the cache is small. For example, at 1% of the 1M index, MARGO needs nearly 3 times the budget to read as few pages. On the 21M index, HyperPage also cuts the search time per step by up to 10.4%. The saving holds for a second agent model and a second corpus.**

***Index Terms*—vector search, disk-resident index, physical design, page layout, workload logs, agent search**

I. Introduction

Vector databases answer approximate nearest neighbor (ANN) queries over collections that exceed main memory. The dominant single-node design keeps product-quantization (PQ) codes [1] in DRAM and stores each vector with its adjacency list in a proximity graph as one record on SSD [2]. A page holds B records, and beam search needs the page of every node it expands. Which records share a page changes no answer, but it changes the page reads, so the page layout is a physical-design decision of the vector database. It matters most when DRAM is small next to the index. In our record format, a billion records fill about 683 GB of pages, so a workstation with 64 GB of DRAM [2] caches under 5% of the index after 32 GB of PQ codes.

Corresponding author: Weiwei Sun.

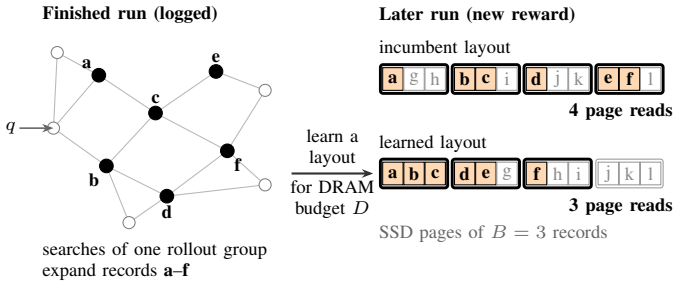

Fig. 1. The task, on a constructed example with $B = 3$. The logs of a finished run decide which records share an SSD page for a DRAM budget D. With a read-once buffer, a later group that expands a to f reads 4 pages under the incumbent layout and 3 under the learned one.

Search agents in training issue such queries at scale. An agent trained with group-relative policy optimization (GRPO) [3], [4] runs for hundreds of steps, each rolling out 128 prompts eight times, and every search call can be logged with the graph nodes it expands. A later run searches the same corpus with the same tool under a new reward, and its traversals overlap heavily. On the run we test on, 83.9% of the node expansions of a rollout group repeat a node that the group has already expanded.

We ask whether the logs of a finished run can lay out the index so that later runs read fewer pages under a declared DRAM budget (Fig. 1). A layout refined once then serves every later run of a retrieval tier shared by many runs and agents. A layout acts only on the accesses left uncached, so it stands in for budget. We study caches of 1% to 25% of indexes of 1M and 21M records. At equal shares, a larger index caches more of the workload: 1% of the records serve 24.6% of the test accesses at 1M and 36.4% at 21M.

Existing layouts use no logs or logs of another workload. Starling and MARGO derive co-location scores from the graph and the vectors [5], [6], and MARGO's weights assume that queries follow the distribution of the indexed vectors. Workload hypergraph partitioning [7], [8] treats logged requests as independent lookups. None of these works bounds the reads of the best layout from below, so none can say how much a better layout could still save.

Learning a layout from logs meets three obstacles. First, the logs must stay valid under every candidate layout, which page search breaks because its walks depend on the layout.

Second, the saving depends on the buffer and the budget. Stock DiskANN reads once per expanded node, a cache of the whole index reads each page once under any layout, and caching hot records shrinks the gap between layouts. Third, the best layout is NP-hard to find once a page holds three or more records. Swap-based search can stop a factor of $B/2$ from the optimum, so a measured saving does not show how far a layout is from the best one.

Our key observation is that DiskANN-style search routes on in-memory codes and on the adjacency lists of the records it expands, never on their page-mates. The expanded sets logged in one run therefore price every layout exactly under read-once buffers, and under a shared cache in the logged order. This removes the first obstacle. It also shows that records held in a memory cache add no page reads, wherever they sit. For the second obstacle, HYPERPAGE builds layouts for a declared budget on the logged unions of the uncached records. It then replays validation steps of the next run to select the layout and the split of the budget between hot records and a page LRU (Fig. 2). For the third, a per-incidence Lagrangian relaxation bounds the reads of every layout from below, so a measured layout can be placed against the optimum.

We train on one run of a Search-R1 agent and test on 16,640 rollout groups of a later run with another reward, whose prompts never occur in training. HYPERPAGE reads fewer pages than MARGO at every budget, 16.4% to 21.2% fewer at 1M and 5.2% to 16.9% fewer on the full corpus, and no compared layout reads fewer. At 1% of either index, MARGO needs 1.5 to 3.0 times the budget to read as few pages. In a 12-thread DiskANN-based engine with equal measured memory, HYPERPAGE cuts the median search time per step by up to 10.38% on the full corpus. This engine with HYPERPAGE's layout is also faster than MARGO's page-search engine with either layout. The saving is largest at small cache shares. When the cache serves most accesses, a refinement that ignores the budget reads more pages than MARGO, so the budget must enter refinement. The saving holds for a second agent model and a second corpus.

Our contributions are as follows.

- **A replay model that prices every layout from one run's logs.** We state conditions under which the page reads of a buffer scope equal the connectivity of the union of its logged expanded sets, for every layout and arrival order. A patched DiskANN issues exactly the replayed reads.
- **Cache-aware refinement and selection for a declared budget.** HYPERPAGE builds layouts on the unions of the uncached records and selects the layout and cache split by replay on validation steps, with MARGO, Starling and MARGO's partitioner on the uncached records among the candidates. It reads fewer pages than MARGO at all seven budgets we test.
- **A map of what can be guaranteed.** Under read-once scopes, every layout reads at most B times the optimum, swap descent can stall at $B/2$, a matching reaches $\lceil B/2 \rceil$, and the problem is APX-hard. A per-incidence Lagrangian bound places the 1M test optimum between 543.2 and 974.4 reads per group. Under a shared LRU no ratio in B exists, and a window bound holds there.

II. Related Work

Layouts and engines of disk-resident graph indexes: Existing layouts of disk-resident graph indexes come from the graph and the vectors and target single queries. HYPERPAGE instead learns a fixed graph's layout from a finished run's traversals and prices each candidate exactly under a declared buffer and budget. Starling shuffles records into blocks to maximize the in-block share of out-neighbors and proves the problem NP-hard with no finite-factor approximation [5]. MARGO weights arcs by the monotonic search paths they support, fills pages greedily, and observes that more co-located edges need not mean fewer random I/Os [6]. The SectorPack routine of OOD-DiskANN [9], DiskANN++ [10], PageANN [11], GPS [12], BAMG [13] and VeloANN [14] change the packing, graph, record format or edges, and DGAI [15] and GoVector [16] buffer the pages of a search. Only versions 1 and 2 of the LSM-VEC preprint use queries, reordering a dynamic disk graph under a query-sampled edge heatmap [17] that we port as HEAT.

Batch and agent workloads: Agent search systems change how a batch is searched or cached, not the page layout. CoWalk runs an agent batch as cooperating walks that read each page once and rank every vector of a fetched page [18], so its walks depend on the layout. DeepResearchGym serves deep-research agents from a DiskANN index [19], and InfoAgent and SemDN cache results or chunks [20], [21], which compose with a page layout.

Workload hypergraphs and set objectives: In workload-driven partitioning the application fixes each access set, while HYPERPAGE's sets are unions of graph traversals that stay valid under every layout only when walks do not depend on it. Connectivity partitioning of hypergraphs is classical [22], [23]. Bandana minimizes the mean number of NVM blocks that each logged embedding lookup touches [7], and MaxEmbed places co-appearing embeddings on SSD pages with replication [24]. SHP minimizes a probabilistic fanout of logged queries [8]. Schism partitions a co-transaction graph [25], and Qd-tree [26] and PAW [27] minimize the blocks a predicate workload accesses. We adopt SHP's objective at $p = 0.5$ as P05-G.

Trace-driven clustering for graph traversals: HYPERPAGE extends trace-driven clustering of graphs to ANN walks on a fixed graph, without replication. Object-graph clustering partitions graphs weighted by successive-access probabilities or trace-derived edges [28]–[30], and Nodine et al. bound the speed-up of blocking for external graph search [31]. ICBL and DiskGNN lay out graphs on disk [32], [33], and SlashBurn, Gorder and near-neighbor reordering order vertices in memory [34]–[36].

Workload-driven adaptation of vector indexes: Unlike systems that adapt a vector index to its workload, HYPERPAGE changes only the node-to-page assignment of a fixed graph

index. ConStore co-locates co-retrieved documents in content segments and rewrites them under drift [37]. Other systems adapt partitions, trees or edges [38]–[42], storage requests [43], result caches [44], HNSW page placement [45], tier placement [46] or the graph itself [47] to the workload. Replay prices hypothetical page layouts as what-if calls price index designs [48], here exactly under Assumption 1.

III. Preliminaries and Problem Definition

We write G for the index graph and $\mathcal{H}$ for the workload hypergraph.

Definition 1 (Data, index, records, pages)**.** The dataset is $X = \{\mathbf{x}_v\}_{v \in V} \subset \mathbb{R}^d$ with $n = |V|$. The proximity graph $G = (V, A)$ is fixed, with out-neighborhoods $N^+(v) = \{u : (v, u) \in A\}$ and $|N^+(v)| \le R$. Node v is stored as one record of s bytes that holds $\mathbf{x}_v$ and $N^+(v)$. With c bytes per vector component and L_2 distance, $s = cd + 4(R + 1)$. A page has P bytes and holds $B = \lfloor P/s \rfloor$ records.

Definition 2 (Layout, connectivity)**.** A layout is a map $\pi : V \to [k]$ with $k = \lceil n/B \rceil$ and $|\pi^{-1}(p)| \le B$ for every page p. Π_B denotes the set of such layouts. For $S \subseteq V$, the page counts are $c_{S,p}(\pi) = |S \cap \pi^{-1}(p)|$, the page set is $\pi(S) = \{\pi(v) : v \in S\}$, and the connectivity is $\lambda_\pi(S) = |\pi(S)|$.

Definition 3 (Logged runs, requests, groups)**.** A logged run has numbered steps. In a step, each prompt is rolled out ν times. A rollout proceeds in turns, and in a turn it may issue one tool call with a list of query strings. A *request* is one tool call. A *sub-query* q is one element of its list. A *group* b is the set of sub-queries of one (step, prompt) over its ν rollouts. On our logs $\nu = 8$, a step has 128 prompts, and a request holds 1.68 sub-queries on average.

Definition 4 (Beam search, expanded sets)**.** Sub-query $q \in \mathbb{R}^d$ (a string after embedding) runs best-first beam search from a fixed entry point with candidate-list size L and beam width W, with routing distances from in-memory PQ codes. Round t expands the beam $e_q^{(t)}$ with $|e_q^{(t)}| \le W$. The expanded set is $e_q = \bigcup_t e_q^{(t)}$. Full-precision vectors from fetched records only re-rank the final candidates.

We assume no node cache. A fixed cache C replaces each e_q by $e_q \setminus C$ and changes nothing else. Expanded nodes are marked visited, so the rounds are disjoint. Four assumptions describe the engines for which logged accesses remain valid under a new layout and buffers read each page once.

Assumption 1 (A1: layout-independent walk)**.** For every sub-query q, every round $e_q^{(t)}$ is a function of (q, G, L, W) and the PQ codes alone. It depends neither on π, nor on buffer contents, nor on other sub-queries. A1 holds when routing uses only the in-memory codes and the adjacency lists in the records of expanded nodes. Ties between equal PQ distances must break by original node identifiers, and co-resident records must never enter a candidate list.

Assumption 2 (A2: reads of expanded nodes only)**.** The engine reads whole pages, only pages in $\pi(e_q)$, and holds page $\pi(v)$ when it expands $v \in e_q^{(t)}$. It does not prefetch.

Assumption 3 (A3: cold start)**.** The buffer is empty when its sub-query or scope starts.

Assumption 4 (A4: read-once scope buffer)**.** Within a scope, a page that has been fetched, or is in flight, serves every later request for that page by any sub-query of the scope until the scope ends. There is no eviction within the scope, and concurrent requests for an in-flight page are coalesced into one read.

Definition 5 (Engines and buffer scopes)**.** E_{node} reads one page per expanded node, so $\mathrm{IO}_{\mathrm{node}}(\pi; q) = |e_q|$. E_{buf} keeps every fetched page until the sub-query ends and coalesces reads in flight. A *scope* σ is a set of sub-queries, such as one request, one group b, one turn across a step, one step, or one session. The scoped buffer S_σ satisfies A4 for σ, and the sub-queries of σ walk independently. L_D is one LRU page cache of D pages shared by all sub-queries in a given arrival order. E_{ps} is page search. Every record of a fetched page is scored and may enter the candidate list, so $e_q = e_q(\pi)$ and A1 fails.

Definition 6 (Keyed buffer, proposed)**.** The keyed buffer is S_b for groups: the server keeps one page buffer per key (step, prompt) for the generation phase of that step, which every request must carry.

Definition 7 (Memory budget, hybrid cache)**.** A deployment declares a budget of D pages of DRAM for index data, a share $\eta = D/k$ of the index. PQ codes stay in DRAM in every compared setting and are not charged to D. For a hot-record share $\rho \in [0, 1]$, the hybrid cache $\mathsf{H}_{D,\rho}$ spends $\lfloor \rho D \rfloor$ pages on a static cache of the $B\lfloor \rho D \rfloor$ records most frequent in training, whose accesses read nothing. Its other pages form one LRU page cache shared by all sub-queries in a given arrival order.

$\mathsf{H}_{D,0}$ is L_D. At $\rho = 1$ every uncached access reads its own page, so every layout reads the same. A pure page cache of $D \ge k$ pages reads each touched page once, and a split cache does so when the residual pages fit in its page part. We report η of 1, 5 and 25%, and on the full corpus also the 2 GiB memory limit of MARGO's evaluation [6], which is 15% of that index.

Definition 8 (Scope hypergraph and objective)**.** A training workload is a set Σ_{tr} of scopes. Each scope σ has the union $E_\sigma = \bigcup_{q \in \sigma} e_q$. Identical unions are merged, and $\omega_E > 0$ is the normalized multiplicity of E, with $\sum_E \omega_E = 1$. The scope hypergraph is $\mathcal{H} = (V, \mathcal{E}, \omega)$ with $\mathcal{E} = \{E_\sigma : \sigma \in \Sigma_{\mathrm{tr}}\}$. Single-sub-query scopes give the per-query hypergraph. We define

$$F_{\mathrm{conn}}(\pi) = \sum_{E \in \mathcal{E}} \omega_E \big(\lambda_\pi(E) - 1\big). \tag{1}$$

Problem 1 (Learned page layout of a fixed disk-resident proximity graph)**.** *Input:* the fixed graph G and record format,

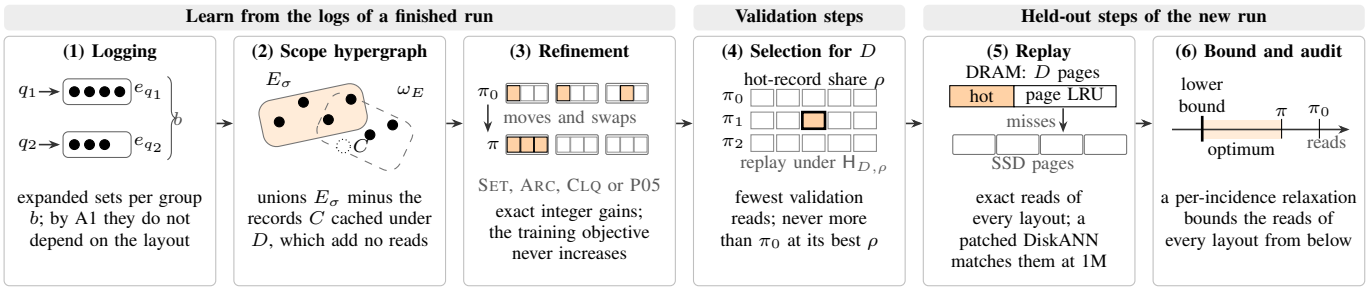

Fig. 2. Overview of HYPERPAGE. Components (1) to (3) learn candidate layouts from the logs of a finished run. Component (4) selects a layout and a hot-record share on validation steps of the new run, and (5) and (6) price the deployment on held-out steps and bound the reads of every layout from below.

the page capacity B, and the weighted unions $\mathcal{E}$ of a finished run. *Output:* a layout $\pi \in \Pi_B$. *Objective:* $\min_{\pi\in\Pi_B} F_{\mathrm{conn}}(\pi)$. *Constraints:* π is a partition, so no record is replicated. The graph, record format, codes and search parameters stay unchanged. *Evaluation target:* the page reads of a later run, at stated buffer scopes and memory budgets. For a declared budget, Section VI-D replaces $\mathcal{E}$ by the unions of the uncached records.

IV. COST MODEL: WHAT A BUFFER READS

The objective of Problem 1 is useful only if it equals what an engine pays. Every page-read number in this paper is attached to one engine through the next lemma, its corollary, or an explicit replay.

Lemma 1 (Per-query cost model)**.** *Let A1, A2 and A3 hold for sub-query q. For every $\pi \in \Pi_B$, E_{buf} reads $\mathrm{IO}_{\mathrm{buf}}(\pi; q) = \lambda_\pi(e_q)$ pages, and $\lceil |e_q|/B \rceil \le \lambda_\pi(e_q) \le |e_q|$. No engine that satisfies A1 to A3 reads fewer pages for q. The answers returned at fixed (L, W) are the same under every layout.*

Proof. By A1 the rounds are the same for every layout. By A2 and A3 every page of $\pi(e_q)$ is read at least once. E_{buf} reads a page at its first reference and never again, so it reads $|\pi(e_q)|$ pages. A page holds at most B nodes of e_q, which gives the lower bound. The candidate list depends only on PQ distances and e_q, and re-ranking uses the vectors of e_q, both fixed by A1. □

The same argument extends to scoped buffers once reads are coalesced within the scope.

Corollary 1 (Scoped buffers)**.** *Let A1 and A2 hold for every sub-query of a scope σ, and let A3 and A4 hold for its buffer. (a) S_σ reads exactly $\lambda_\pi(E_\sigma)$ pages for every π and every arrival order of the sub-queries of σ. No engine that satisfies A1 to A3 for the scope reads fewer. (b) $\lambda_\pi(E_\sigma) \le \sum_{q\in\sigma} \lambda_\pi(e_q)$. (c) With an LRU buffer of $D \ge \lambda_\pi(E_\sigma)$ pages owned by σ alone, in place of the unbounded buffer, the reads are still $\lambda_\pi(E_\sigma)$. (d) Dropping a sub-query whose string repeats one of σ changes neither E_σ nor the reads.*

Proof. (a) By A1 each e_q is the same whether or not other sub-queries run, and in any order. By A4 each page of $\pi(E_\sigma) = \bigcup_q \pi(e_q)$ is read at most once, and by A2 and A3 at least once. (b) A union of page sets is no larger than the sum of their sizes. (c) At most $\lambda_\pi(E_\sigma) \le D$ distinct pages are ever referenced, so LRU never evicts. (d) By A1 equal strings have equal expanded sets, so the union is unchanged. □

Corollary 1 turns logged traversals into exact reads for every candidate layout, so layouts can be compared without running an engine. Part (a) needs no simultaneity, so a keyed buffer stays exact under any interleaving of a group's requests. Part (c) does not cover a cache shared by interleaved scopes. For L_D no such identity holds, because its reads depend on the arrival order. Replay is still exact under A1 to A3 for any fixed order, and at $D \ge k$ nothing is evicted, so every layout reads each touched page once. Lemma 1 does not apply to E_{ps}, where the expanded sets depend on π. Engines that overlap expansion with outstanding reads, such as PipeANN [49], relax the best-first order of expansions, and A1 does not hold for them either. Thus Lemma 1 covers stock DiskANN as E_{node} and DGAI's buffer [15] as E_{buf}, and Corollary 1 covers the proposed keyed buffer and the batch cache of CoWalk [18] on one thread. The page search of Starling, MARGO and CoWalk's cooperative walks [5], [6], [18] is E_{ps}.

When a layout can matter: In the DiskANN main branch of May 2026, every frontier node receives its own read request, and on 10K test queries each of SIFT1M, DEEP1M and T2I-1M issued reads equal expanded nodes. A buffer such as E_{buf} is therefore the precondition for any layout effect, including the 9% that OctopusANN reports from page shuffle [50]. A layout also needs more than one record per page: a 384-dimensional int8 record takes 644 bytes, which gives $B = 6$ at 4 KB and $B = 12$ at 8 KB, the format we study.

V. ANALYSIS: WHAT READS COUNT

By Corollary 1, reads equal connectivity, so minimizing F_{conn} is the exact training cost under a scoped buffer. MARGO and Starling maximize weighted counts of co-located pairs. A pairwise objective can also be built from the logged sets themselves, which separates the information used from the form of the objective.

Definition 9 (Pairwise co-location objectives)**.** For symmetric weights $w : \binom{V}{2} \to \mathbb{R}_{\ge 0}$, let $F_w(\pi) = \sum_{\{u,v\}} w_{uv} \mathbf{1}[\pi(u) = \pi(v)]$, to be maximized. The clique expansion of the logged sets has $w^{\mathrm{clq}}_{uv} = \sum_{E\ni u,v} \omega_E$ (CLQ). Its arc-restricted version

is $w^{\mathrm{arc}}_{uv} = w^{\mathrm{clq}}_{uv} \cdot \mathbf{1}[(u,v) \in A \vee (v,u) \in A]$ (ARC). Starling's objective uses the arc indicator as w, and MARGO's uses $w(u,v) = m(u,v)\, m(u)$ over arcs, where m counts monotonic search paths [6]. Minimizing F_{conn} is the set objective (SET). For a set E, let $\phi_E(\pi) = \sum_p \binom{c_{E,p}(\pi)}{2}$, so that $F_{w^{\mathrm{clq}}}(\pi) = \sum_E \omega_E \phi_E(\pi)$.

The set objective credits a page linearly in the members it holds, and the clique objective credits it quadratically. The following identity makes the difference exact.

Proposition 1 (Set credit plus concentration bonus)**.** *For every layout π and every weighted family of sets, let $g(\pi) = \sum_E \omega_E(|E| - \lambda_\pi(E))$. Then*

$$F_{w^{\mathrm{clq}}}(\pi) = g(\pi) + \Omega(\pi), \tag{2}$$
$$\Omega(\pi) = \sum_E \omega_E \sum_{p \in \pi(E)} \binom{c_{E,p}(\pi) - 1}{2} \geq 0,$$

and $\Omega(\pi) = 0$ if and only if no page holds three or more members of one set with $\omega_E > 0$.

Proof. Apply Pascal's rule $\binom{c}{2} = (c-1) + \binom{c-1}{2}$ to $c = c_{E,p}$ and sum over $p \in \pi(E)$. □

The credit g counts the reads a layout saves against one page per member. Clique weight is also the first layout-dependent term of SHP's probabilistic fanout at small retention θ [8]. Its case of two records per page shows where the form of the objective cannot matter.

Corollary 2 (Two records per page)**.** *If $B = 2$, then $F_{\mathrm{conn}}(\pi) = \sum_E \omega_E(|E| - 1) - F_{w^{\mathrm{clq}}}(\pi)$ for every $\pi \in \Pi_2$. A layout minimizes F_{conn} if and only if its pages form a maximum-weight matching of the clique-expansion graph, completed by pairing unmatched nodes, which takes polynomial time [51].*

Proof. At $B = 2$ every $c_{E,p} \leq 2$, so $\Omega \equiv 0$ and $F_{\mathrm{conn}} = \sum_E \omega_E(|E| - 1) - g = \sum_E \omega_E(|E| - 1) - F_{w^{\mathrm{clq}}}$. The co-located pairs of a layout in Π_2 form a matching of weight $F_{w^{\mathrm{clq}}}(\pi)$, and any matching extends to a layout by pairing unmatched nodes, which cannot decrease the nonnegative weight. □

Corollary 2 gives an exact optimizer at $B = 2$, against which we calibrate the lower bound of this section.

Beyond two records per page, the bonus is limited by the members of one set that fit on a page, which the next proposition uses.

Proposition 2 (Size and capacity bound)**.** *Let $\bar{b} = \max_E \min(|E|, B)$ over the sets with $\omega_E > 0$. For every $\pi \in \Pi_B$, $g(\pi) \leq F_{w^{\mathrm{clq}}}(\pi) \leq \frac{\bar{b}}{2} g(\pi)$. If $\pi_c \in \arg\max F_{w^{\mathrm{clq}}}$ and $\pi^\star \in \arg\min F_{\mathrm{conn}}$ over Π_B, then $g(\pi_c) \geq \frac{2}{\bar{b}} g(\pi^\star)$ and $F_{\mathrm{conn}}(\pi_c) - F_{\mathrm{conn}}(\pi^\star) \leq (1 - \frac{2}{\bar{b}}) g(\pi^\star)$.*

Proof. For $1 \leq c \leq \bar{b}$, $c - 1 \leq \binom{c}{2} \leq \frac{\bar{b}}{2}(c-1)$. Summing gives the two-sided bound, and $g(\pi_c) \geq \frac{2}{\bar{b}} F_{w^{\mathrm{clq}}}(\pi_c) \geq \frac{2}{\bar{b}} F_{w^{\mathrm{clq}}}(\pi^\star) \geq \frac{2}{\bar{b}} g(\pi^\star)$. □

The clique objective is thus a $2/\bar{b}$ relaxation of the page savings of exact maximizers, with $\bar{b} = B$ on sets larger than a page. It says nothing about the heuristic arms, MARGO's weights or held-out scopes. The next result shows that the factor cannot be raised.

Proposition 3 (Tightness)**.** *For every $B \geq 3$ and $\varepsilon > 0$ there is an instance on which $F_{w^{\mathrm{clq}}}$ has a unique maximizer π_c, up to renaming pages, and $\frac{2}{B} \leq g(\pi_c)/g(\pi^\star) \leq (1+\varepsilon)\frac{2}{B}$, where $\pi^\star$ minimizes F_{conn}.*

Proof sketch. On $V = [B] \times [B]$, take the rows as sets of weight $1 + \varepsilon$ and every pair of cells in one column as a set of weight 1. The row layout uniquely maximizes $F_{w^{\mathrm{clq}}}$ with $g = (1+\varepsilon)B(B-1)$, and the column layout has $g \geq B\binom{B}{2}$. □

The scope is a second axis. A per-query objective, set or pairwise, scores each sub-query of a scope on its own.

Proposition 4 (Cross-query reuse)**.** *For a scope b and a layout π, let $r_{b,p}(\pi) = |\{q \in b : p \in \pi(e_q)\}|$. Then*

$$\sum_{q \in b} \lambda_\pi(e_q) = \lambda_\pi(E_b) + \sum_{p \in \pi(E_b)} \big(r_{b,p}(\pi) - 1\big). \tag{3}$$

There exist workloads with $B = 2$ for which every layout that minimizes the per-query objective $\sum_b \sum_{q \in b} \lambda_\pi(e_q)$ reads strictly more scoped pages $\sum_b \lambda_\pi(E_b)$ than some other layout.

Proof. Each page of $\pi(E_b)$ is counted once on the right of (3) by $\lambda_\pi(E_b)$ and $r_{b,p} - 1$ more times by the sum, and once on the left for every sub-query of b that touches it. For the inversion, take $B = 2$, a scope with the set $\{v, z\}$, and a scope with the sets $\{u, v\}$ and $\{w, z\}$. The pages $\{u, v\}, \{w, z\}$ cost 4 per query and 4 scoped, the unique per-query optimum, while $\{u, w\}, \{v, z\}$ cost 5 and 3. □

The reuse term of (3) is what a scoped buffer saves, and it depends on the layout. Proposition 4 is why HYPERPAGE trains on scope unions: an objective that scores the sub-queries of a scope separately can miss that reuse.

Exact optimization is out of reach beyond two records per page, which is why we refine heuristically and audit the result with a lower bound.

Proposition 5 (Complexity)**.** *Over arbitrary weighted set families, minimizing F_{conn} on Π_B is NP-hard for every fixed $B \geq 3$, and it takes polynomial time for $B = 2$.*

Proof. The case $B = 2$ is Corollary 2. For $B \geq 3$, take a graph $H = (U, E_H)$ with $|U| = Bk$ and use its edges as sets. Every page then holds B nodes, and F_{conn} counts cut edges, up to the normalization of ω. The cut is at least $|E_H| - k\binom{B}{2}$, with equality if and only if every page induces K_B. Deciding whether such a K_B-factor exists is NP-complete [52]. □

Approximation is limited as well. For every fixed $B \geq 3$, minimizing the reads of a read-once workload is APX-hard even on weighted pairs, so no polynomial approximation scheme exists unless P = NP. The reduction is from maximum independent set on 3-regular graphs [53], [54].

TABLE I
WORST-CASE READS AGAINST THE OPTIMUM $R^\star$ UNDER READ-ONCE SCOPES.

Layout	Reads at most
Any layout (tight), so also Algorithm 1	$(U/L_{\mathrm{cap}})R^\star \le BR^\star$
Some swap-local optima of R, every B	$\frac{B}{2}R^\star$ reached
Maximum-weight matching, polynomial	$\lceil B/2\rceil R^\star$
Exact maximizer of $F_{w^{\mathrm{clq}}}$ (from Proposition 2)	$\lfloor(\bar{b}+1)^2/4\rfloor R^\star/\bar{b}$
Exact minimizer of the fanout at $\theta=\frac{1}{2}$	$(2-2^{1-B})R^\star$
No polynomial algorithm, unless P = NP	$(1+\varepsilon_B)R^\star$

What can be guaranteed: Write $R(\pi)=\sum_E \omega_E\lambda_\pi(E)$ for the reads of a read-once workload and $R^\star$ for their minimum over Π_B. Page capacity alone bounds every layout.

Proposition 6 (Capacity guarantee)**.** *Let* $L_{\mathrm{cap}} = \sum_E \omega_E\lceil|E|/B\rceil$ *and* $U = \sum_E \omega_E \min(|E|,k)$. *Every* $\pi\in\Pi_B$ *satisfies* $L_{\mathrm{cap}} \le R^\star \le R(\pi) \le U \le B\,L_{\mathrm{cap}}$, *and the factor* B *is attained.*

Proof. Per set, $\lceil|E|/B\rceil \le \lambda_\pi(E) \le \min(|E|,k) \le B\lceil|E|/B\rceil$. On B disjoint sets of B records, one set per page reads B, and one record of each set per page reads B^2. □

Table I shows where HYPERPAGE stands. Its refinement is swap descent, so it inherits only the first row. Swaps can stall at a factor of $B/2$. With $\lfloor B/2\rfloor$ disjoint sets of $2B$ records on B pages, two records of every set on every page read $B/2$ times the optimum, and no exchange lowers the reads. Polynomial time reaches $\lceil B/2\rceil$: a maximum-weight matching of the clique expansion [51], packed $\lfloor B/2\rfloor$ pairs to a page, outweighs a random matching inside the pages of an optimal layout. The clique expansion of our unions does not fit in memory, so the matching stays a reference point. The surrogates differ sharply. The fanout credits a page between $\frac{1}{2}$ and $1-2^{-B}$ per touched set, so its exact minimizer stays within a factor of two at every B, while the factor of the clique objective grows with B. This agrees with P05 reading less than CLQ at both scales in our experiments. For fixed $B\ge 3$ the best polynomial factor therefore lies between $1+\varepsilon_B$, a small constant, and $\lceil B/2\rceil$, and this range is open. For a measured layout we instead use the ratio to a lower bound: any $L\le R^\star$ gives $L\le R^\star\le R(\pi)=\varrho(\pi)L$, so π is within $\varrho(\pi)$ of the optimum. Next is a bound L stronger than L_{cap} on our workloads. Let $\beta=B-1$, $h(t)=1/(1+t)$, $m_E=\min(\beta,|E|-1)$, and $\psi_E(g)=\min_{t\in\{0,\dots,m_E\}} h(t)+gt$.

Proposition 7 (A lower bound valid for every layout)**.** *Partition the records into classes* $C_1,\dots,C_r$ *and write* $c(u)$ *for the class of* u. *For real* γ_{vE}, *one per record* v *and set* $E\ni v$, *and for* $\mu\in\mathbb{R}^r_{\ge 0}$, *let* $w^\gamma_{vu}=\sum_{E\ni u,v}\omega_E\gamma_{vE}$. *Then for every* $\pi\in\Pi_B$ *the reads* $F_{\mathrm{conn}}(\pi)+\sum_E\omega_E$ *satisfy*

$$\sum_E \omega_E\lambda_\pi(E) \ge \sum_v \Big[\sum_{E\ni v}\omega_E\psi_E(\gamma_{vE}) - \max_{\substack{T\subseteq V\setminus\{v\}\\ |T|\le\beta}} \sum_{u\in T}\big(w^\gamma_{vu}-\mu_{c(u)}\big)\Big] - \beta\sum_i \mu_i|C_i|.$$

Proof. Let S_v be the page-mates of v and $t_E=|S_v\cap E|$, so $c_{E,\pi(v)}=1+t_E$ and $\sum_E\omega_E\lambda_\pi(E)=\sum_v r_v$ for $r_v=\sum_{E\ni v}\omega_E h(t_E)$. As $t_E\le m_E$, $h(t_E)\ge\psi_E(\gamma_{vE})-\gamma_{vE}t_E$, and $\sum_{E\ni v}\omega_E\gamma_{vE}t_E=\sum_{u\in S_v}w^\gamma_{vu}$. Hence $r_v\ge\sum_{E\ni v}\omega_E\psi_E(\gamma_{vE})-\sum_{u\in S_v}(w^\gamma_{vu}-\mu_{c(u)})-\sum_{u\in S_v}\mu_{c(u)}$, and as $|S_v|\le\beta$ the middle sum is at most the maximum over T. Page-mates are mutual, so $\sum_v\sum_{u\in S_v}\mu_{c(u)}=\sum_i\mu_i\sum_{u\in C_i}|S_u|\le\beta\sum_i\mu_i|C_i|$. □

As in Lagrangian relaxation [55], each γ_{vE} prices one page-mate of v inside E, so the bound sees how a record's page-mates spread over its sets. The prices μ charge each class for its page-mate capacity $\beta|C_i|$, a constraint aggregation [56], and the inner maximum needs only each record's β heaviest partners per class. We use degree classes and projected subgradient steps. The worst-case gap grows with B, because the proof also holds for fractional page-mates that need not be mutual. On the vertex stars of a δ-regular graph of girth above B with $\delta\ge(B+1)/2$ [57], whose edges are the records, spreading each record evenly over its $2\delta-2$ neighbours costs at most $4(B+1)/(B(B+2))$ per record, while a page of j records touches at least $j+1$ sets. The optimum thus exceeds every bound of Proposition 7 by a factor of at least $(B+2)/4$.

Shared caches: None of this transfers to a page LRU, whose misses depend on the order of accesses. With $B=2$ and one LRU page, the trace $(a,b)^T$ misses once on the pages $\{a,b\},\{c,d\}$ and $2T$ times on $\{a,c\},\{b,d\}$. No function of B therefore bounds the ratio of two layouts, and the sum of cold scope bounds does not bound the misses either. A weaker bound survives.

Proposition 8 (Window bound)**.** *Fix a record cache and a page cache of* H *pages, empty at the start. Cut the access trace into consecutive windows with uncached record sets* $S_1,\dots,S_J$. *Every layout misses at least* $\lceil|S_1|/B\rceil+\sum_{j\ge 2}\max(0,\lceil|S_j|/B\rceil-H)$ *times.*

Proof. Window j touches at least $\lceil|S_j|/B\rceil$ pages, and at most H of them are resident when it starts. □

VI. HYPERPAGE

Fig. 2 shows the six components of HYPERPAGE. Section VI-A covers (1) and (2), Sections VI-B and VI-C cover (3), and Section VI-D adds the budget and covers (4). Section VI-E covers (5), and Proposition 7 gives the bound of (6).

A. Logging and the Scope Hypergraph

For each training sub-query, the logger records the expanded set e_q in expansion order and the group that issued it. By A1 the expansion depends only on the codes and the graph, so the logs are valid for every layout. The optimizer receives the unions E_σ of the training scopes, merged with multiplicity. A group union summarizes a log and assumes nothing about how requests arrive, so a layout trained on group unions can serve any scope and any cache.

Algorithm 1 Refinement$(F, \mathcal{H}, \omega, B, \pi_0, \tau, \epsilon)$

Input: objective F to minimize, weighted unions $\mathcal{E}$, capacity B, incumbent π_0, passes τ, tolerance ϵ
Output: layout $\pi \in \Pi_B$

$\pi \leftarrow \pi_0$; $O \leftarrow$ random order of the covered nodes
for $i = 1, \ldots, \tau$ **do**
  for all $v \in O$ **do**
    $\mathcal{P} \leftarrow$ the $\bar{p}$ candidate pages of v
    among moves into free slots of $\mathcal{P}$ and swaps with occupants of $\mathcal{P}$, take the most negative exact change ΔF
    apply it if $\Delta F < 0$
  end for
  break if the pass gain is at most $\epsilon F(\pi)$
end for
return π

B. Objectives and Exact Gains

Every objective is a sum over sets of a function of the page counts $c_{E,p}$. SET minimizes F_{conn}, CLQ and ARC maximize the pairwise objectives of Definition 9, and P05 minimizes SHP's probabilistic fanout [8] at $\theta = \frac{1}{2}$, $\sum_E \omega_E \sum_{p \in \pi(E)} (1 - 2^{-c_{E,p}})$. With integer multiplicities and a scale of 2^{12}, its gains are exact integers for $B \leq 12$. Moving v from page p to page p' changes F_{conn} by

$$\Delta(v; p \to p') = \sum_{E \ni v} \omega_E \big(\mathbf{1}[c_{E,p'} = 0] - \mathbf{1}[c_{E,p} = 1] \big). \quad (4)$$

The first term counts sets that gain page p', and the second counts sets that lose page p. A swap of u on p' and v on p leaves the counts of every set containing both nodes unchanged, so its exact change is

$$\Delta(u \leftrightarrow v) = \Delta(v; p \to p') + \Delta(u; p' \to p) + \sum_{E \ni u, v} \omega_E \big(\mathbf{1}[c_{E,p} = 1] + \mathbf{1}[c_{E,p'} = 1] \big). \quad (5)$$

The last sum cancels the terms that the two single moves charge to such sets. The other objectives have the same structure: a move changes only the counts of the sets that contain the moved record, and a swap corrects the sets that contain both records.

C. Refinement from the Incumbent

Algorithm 1 starts from the incumbent, which transfers better than a start from scratch. Candidate pages of v are the pages of its co-members within 32 positions in up to 64 of its hyperedges, plus the pages of its graph neighbors inside training hyperedges. We keep the $\bar{p} = 16$ pages with the most such co-members. Covered nodes initiate moves, and any record on a candidate page can be a swap partner. The search is strict descent: in a fixed random node order it applies the best move or swap, by exact gain under the hard capacity, only if $\Delta F < 0$. Unlike Fiduccia and Mattheyses [58], it makes no uphill moves and no rollback, and with its candidate pages, pass limit and tolerance it certifies no local optimum. One pass takes $O(\bar{p} B (\text{nnz} + n \deg_{\max}))$ time and $O(\text{nnz} + n)$ memory, where $\text{nnz} = \sum_E |E|$ and $\deg_{\max}$ is the largest number of sets that contain one node.

Proposition 9 (Monotone refinement)**.** *Algorithm 1 returns π with $F(\pi) \leq F(\pi_0)$. For $F = F_{\text{conn}}$, the mean reads per training scope under the scoped buffer do not exceed those of the incumbent.*

Proof. Each applied operation has a negative exact change, so F decreases with every operation. Corollary 1 gives mean training reads $F_{\text{conn}} + 1$. □

Proposition 9 rules out regressions on training scopes only. Selection on validation scopes of the new run, defined in Section VI-D, rules them out on the scored validation steps and promises nothing beyond them.

D. Refinement and Selection for a Declared Budget

A declared budget fixes which records the cache will hold, and the training objective should not reward placing them. For a budget of D page slots and a hot-record share ρ, the hybrid cache $\mathsf{H}_{D,\rho}$ holds the set C of the $B\lfloor \rho D \rfloor$ records most frequent in training (Definition 7).

Corollary 3 (Reads under a record cache)**.** *Let A1 to A4 hold for a scope σ, and let a fixed set C of records be served from memory. Then S_σ reads exactly $\lambda_\pi(E_\sigma \setminus C)$ pages for every $\pi \in \Pi_B$ and every arrival order.*

Proof. A fixed cache replaces each e_q by $e_q \setminus C$ and changes nothing else, so Corollary 1(a) applies to the sets $e_q \setminus C$, whose union is $E_\sigma \setminus C$. □

By Corollary 3, a record of C adds nothing to the reads of any scope, wherever it is placed. Cache-aware refinement therefore runs Algorithm 1 on the residual unions $E \setminus C$ with their weights. Records of C then belong to no set. They never initiate a move, and their page slots serve as free partners for swaps among uncached records. A second variant also freezes the records outside the training unions on the incumbent's pages, because the logs carry no evidence about them. We set ρ to the share that the incumbent chooses on validation steps, which is known before refinement.

The misses of the page LRU of $\mathsf{H}_{D,\rho}$ depend on the arrival order, which the unions discard. Selection closes this gap with trace replay. For a declared D, every candidate layout, the incumbent included, is replayed under $\mathsf{H}_{D,\rho}$ in the deployment's arrival order, which the engine can log, on validation steps of the new run for every share ρ of a grid. The deployment is the pair of layout and share with the fewest validation reads, with ties going to the incumbent. The incumbent with its best share is always a candidate, so the selected pair never reads more validation pages than the incumbent under the same budget.

E. Replay Engine

The engine patches stock DiskANN and keeps its record format, so every layout shares one engine. A scoped buffer is

TABLE II
TEST WORKLOADS. RECORDS ARE 644 BYTES, SO $B = 12$ AT 8 KB. OVERLAP IS $1 - \sum_b |E_b| / \sum_q |e_q|$.

Workload	units	sub-queries	$\lvert e_q \rvert$	$\lvert E_b \rvert$	overlap	L	recall@10
W3-1M (main)	16,640	467,989	512.1	2,312	83.9%	500	0.946
W3-21M	16,640	467,989	818.4	4,475	80.6%	800	0.949
W2	15,360	590,160	512.1	2,068	89.5%	500	0.950
W1	29,256	963,624	1,211.4	11,830	70.4%	1,200	0.956
W4 (proxy)	41,355	1,078,138	513.8	1,858	86.1%	500	0.874
DEEP1M	7,174	199,998	49.1	1,264	7.6%	35	0.960

a hash map from page to an aligned page slot, emptied when the scope ends. An expanded node is a hit if the scope has read its page, a shared read if the round has already requested it, and otherwise one page read. Hits and misses stay in frontier order and no co-resident record is scored, so A1 holds, and Corollary 1 gives the reads. A relayout writes the records in the order of π and stores a map from node to page and slot, so identifiers, PQ codes and the entry point are identical across layouts. For read-once scopes, threads take whole groups and run their logged sub-queries in order. For the hybrid cache $\mathsf{H}_{D,\rho}$, the hot records enter DiskANN's node cache, and every other page passes through one LRU page cache shared by all threads. A page stays pinned while a round uses it, and a request for a page in flight waits for that read. Sub-queries arrive turn by turn, and threads take them dynamically. With one thread, the engine performs the replay's cache operations in the replay's order.

VII. EXPERIMENTS

We ask when a learned layout reads fewer pages than MARGO under an equal cache budget at both scales (**RQ1**), which design choices produce the saving (**RQ2**), and how far the layouts are from the optimum (**RQ3**).

A. Experiments Setup

1) Datasets: Table II lists the workloads. The main workload W3 comes from two public runs of the Search-R1 task [4][1] with Qwen3-8B. They search one corpus with one tool and differ in reward: an outcome reward and a process reward from an LLM verifier. Training uses steps 241 to 400 of the outcome run, in three draws by step. Validation uses steps 41 to 70 of the process run (3,836 groups), and test uses steps 71 to 200 (16,640 groups). Prompt sets align between the runs on all 400 steps, and no validation or test prompt occurs in a training step. We replay W3 on two indexes of the 21,015,324 DPR passages [60] that the task retrieves from. W3-1M indexes a uniform sample of 1M passages, and W3-21M indexes all of them in 1,751,277 pages, 13.36 GiB at 8 KB. The retrieval server of these runs is not public, so we reissue every query string of both runs through our disk-resident index and log the expanded sets, which by A1 hold under every layout. The 683 MB index of W3-1M fits in DRAM, so at that scale the budget is a declared operating point.

[1]https://huggingface.co/datasets/wckwan/PRM-agent-rl-artifacts

Four further workloads test the direction of the result: deep-research sessions of OpenResearcher [61] (W1), the outcome run alone (W2), proxy groups from OLMo-3-7B trajectories[1] (W4), and random batches of W3 sizes on DEEP1M [62], [63]. W2 shaped the design of W3, so it does not count as confirmation. All texts are embedded with bge-small-en-v1.5 [64], 384-dimensional and quantized to int8.

2) Metrics: The primary metric is the replayed page reads per test step under the hybrid cache $\mathsf{H}_{D,\rho}$ of Definition 7, kept across the 130 test steps from an empty start. Replay uses turn lockstep, which interleaves the rollouts of a step turn by turn as the official Search-R1 driver does. Each layout gets its own hot-record share ρ from $\{0, 0.1, 0.25, 0.5, 0.75, 0.9, 1\}$: the share with the fewest validation reads after a warm-up of 10 validation steps. The W3 splits and the candidate arms other than P05-G and the 1M ARC-C were fixed before any W3 layout was trained. P05-G was added after the other 1M results were known, and ARC-C at 1M after all of them under an admission rule fixed before it was trained. HEAT's cache-aware layouts, MARGO on the uncached records and STARLING came after all results, the last two in Table III only. Without P05-G, P05-C and the 1M ARC-C, HYPERPAGE saves 19.8%, 14.8% and 13.0% at 1M. Without the three added last, it saves 3.1% and 1.9% at 15% and 25% of the full corpus. For a cache-aware layout, MARGO's validation share fixes the hot set that refinement removes, and selection then uses the same validation steps. The test steps enter neither choice. The secondary metric is the mean reads under read-once scopes S_σ, which Corollary 1 makes exact. Recall is identical for every layout at fixed L, and we choose the smallest L with validation recall@10 of at least 0.95, $L = 500$ at 1M and $L = 800$ on the full corpus. The margin of layout A over MARGO is $M = 1 - R_A/R_{\mathrm{MARGO}}$ for total reads R averaged over the three training draws. A patched DiskANN reproduces the stock results and expansion sequences of 7,850 W3 sub-queries. With one thread, it reads exactly the replayed pages of every 1M validation step under the hybrid cache. On the full corpus, nearties of PQ distances change 44 of 66,375 validation calls, and the engine's read savings stay within 0.1 points of the replay.

We test each margin with a paired bootstrap over the test steps with 2,000 resamples, also with moving blocks of 5, 10 and 26 steps. In Table III, every margin of HYPERPAGE and 85 of the 90 margins of the compared methods differ from zero at level 0.05. Among the other rows, only CLQ-G at 25% of the 1M index and MARGO on uncached records at 5% of the full corpus lose it. At 1M under the keyed group buffer, the margins of SET-G over MARGO and of the first three ablation steps are positive at level 0.05 under a bootstrap over prompts. The text's savings for the 1M candidate ablation, measured memory, the extra full-corpus step windows, W1, W2, smaller pages and the result cache are means of per-draw savings. The other numbers are ratios of means, and neither kind has a test.

3) Compared methods: All layouts share the graph, record format and page size, start from MARGO's page order unless stated, and are replayed on the same buffers and caches. Residual unions drop the records that the budget's cache holds.

TABLE III

SAVING IN PAGE READS OVER MARGO UNDER EQUAL CACHE BUDGETS (%, HIGHER IS BETTER, MEAN OF 3 DRAWS, STD BELOW 0.5). THE FIRST ROW GIVES MARGO'S READS PER TEST STEP. "ON RESIDUAL UNIONS" GIVES A METHOD HYPERPAGE'S CACHE-AWARE INPUT. PROXIMITY AND HEAT ARE OUR PORTS. BOLD MARKS THE BEST PER COLUMN. THE LAST ROW GIVES THE BUDGET, AS A MULTIPLE OF THE DECLARED ONE AND A BRACKET ON THE BUDGET GRID, AT WHICH MARGO WITH ITS VALIDATION SHARE READS AS FEW PAGES AS HYPERPAGE.

	W3-1M			W3-21M (full corpus)				W4-1M (layouts from W3)		
Budget	1%	5%	25%	1%	5%	15%	25%	1%	5%	25%
MARGO [6], reads per step	880.4k	594.1k	220.1k	1,158.0k	561.3k	168.9k	113.8k	817.0k	505.4k	200.6k
STARLING [5]	2.8	2.8	2.7	3.6	5.3	4.5	4.3	1.3	2.1	1.9
RCM [59]	−22.6	−19.3	−16.8	−16.1	−27.1	−20.0	−15.7	−15.1	−15.7	−14.1
Proximity [14]	−3.3	−3.9	−4.3	−8.1	−11.9	−10.7	−9.2	0.1	−0.8	−2.0
FREQ	−21.1	−14.2	−5.4	−19.6	−18.9	−15.4	−11.3	−15.6	−11.2	0.2
SHP-G [8]	0.1	0.0	−0.1	−0.2	−0.3	−0.1	0.0	0.0	0.0	−0.1
Mt-KaHyPar [23]	7.4	1.8	−4.7	−5.1	−10.7	−14.8	−10.7	7.5	4.3	−1.8
on residual unions	9.2	4.9	5.9	−3.9	−5.2	−7.8	−2.2	10.0	1.8	6.7
HEAT [17]	11.6	6.7	3.2	8.6	9.4	−0.1	−2.1	10.5	7.9	4.0
on residual unions	13.4	10.3	11.2	9.6	13.0	**5.2**	2.6	12.3	5.2	10.9
HYPERPAGE	**21.2**	**16.4**	**16.6**	**12.5**	**16.9**	**5.2**	**6.8**	**19.4**	**14.0**	**14.5**
MARGO's budget to match (×)	2.5–3.0	1.4–1.6	1.12–1.16	1.5–1.75	1.15–1.2	≤1.07	≤1.08	2.0–2.25	1.4–1.6	1.12–1.2

- MARGO [6]: monotonic-path arc weights, no logs, from its authors' code[2]. It also runs in its own page-search engine, which breaks A1.
- STARLING [5]: Starling's in-page arcs, refined from the identifier order.
- RCM [59] and Proximity: the reverse Cuthill-McKee order of the graph, and recursive balanced 2-means on the vectors in the spirit of VeloANN [14], both without logs.
- FREQ: records packed by training frequency, our baseline.
- HEAT: our port of LSM-VEC's heatmap [17], logged arc traversals plus one per arc. LSM-VEC specifies neither its weights nor its window and releases no code, so we refine a layout under its arc weights with our optimizer, in the weighting that reads fewer validation pages.
- Mt-KaHyPar [23]: connectivity at exact page capacity, reported on group unions and on residual unions. On group unions it partitions in two levels as in Bandana [7] at 1M. At 21M it refines MARGO's layout with its V-cycles within bands of 43 consecutive pages, because one hypergraph of all pages exceeds 90 GB. On residual unions it takes its variant, one run or two levels, with fewer validation reads.
- SHP-G: SHP's paired moves [8] on the P05 objective, the move rule only.
- Algorithm 1: ARC-G, CLQ-G, SET-G and P05-G under ARC, CLQ, SET and P05 on group unions (cache-blind), ARC-Q and SET-Q on per-query sets, P05-C and ARC-C on the residual unions of one budget (cache-aware), and ARC-CF, which is ARC-C with the records outside the training unions frozen.
- HYPERPAGE: the pair of layout and share with the fewest validation reads among MARGO, STARLING, Algorithm 1's layouts for the budget, and HEAT's objective and MARGO's partitioner on the uncached records.

The layouts of Algorithm 1 share the start, candidate pages, passes and tolerance, so only the objective and the hyperedge family differ. The first cache-aware layout of each scale uses the objective that leads the cache-blind layouts on its validation steps, and ARC-C is also a candidate at 1M.

4) Implementation details: At both scales, the fixed Vamana graph is built with MARGO's DiskANN-derived builder, with $R = 64$, build list size 125 and 32-byte PQ codes. Traversals are logged by stock DiskANN[3] with $W = 4$. Each learned layout is built once per training draw, with at most 8 passes and $\epsilon = 10^{-4}$, and tables report mean ± std over the 3 draws. On one thread, eight passes of a cache-blind layout take 1,991 to 13,830 s, and cache-aware refinement 317 to 5,943 s, because the residual unions keep only 8% to 73% of the incidences. Timing uses one consumer NVMe SSD per host with direct I/O and 12 search threads pinned to their own cores. Both arms of a timed pair hold the same hot records and LRU pages, except at 1% of 1M, where each arm uses its own share. DiskANN's node cache spends 1.25 to 1.72 times the declared share on the hot records of every layout. Our code is publicly available: https://github.com/zhangshea/HyperPage

B. Overall Comparison

Overall: HYPERPAGE reads fewer pages than MARGO in every column of Table III, and no compared layout reads fewer. It saves 16.4% to 21.2% at 1M and 12.5% and 16.9% at 1% and 5% of the full corpus. At 1% and 5%, every layout of Algorithm 1 in Table VII also beats MARGO at both scales, so the logs carry most of the gain when the cache is small. At 15% and 25% of the full corpus, the cache serves most test accesses, and the saving falls to 5.2% and 6.8%.

A layout substitutes for memory: A better layout reaches MARGO's reads with a smaller cache (last row of Table III). MARGO needs 2.5 to 3.0 times the declared budget to read

[2]https://github.com/CodenameYZY/MARGO

[3]https://github.com/microsoft/DiskANN

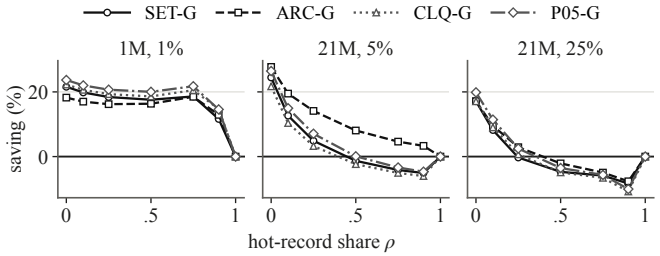

Fig. 3. Page-read saving over MARGO on W3 test when every layout uses the same hot-record share ρ, with no selection (higher is better). Panels give the index and the budget.

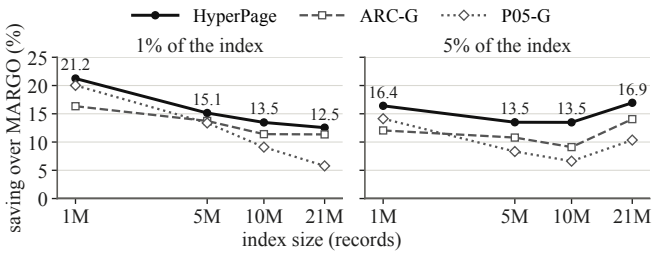

Fig. 4. Saving in page reads over MARGO on W3 test as the index grows (%, higher is better, 3 draws). Every layout uses its own validation share. HYPERPAGE picks among MARGO, ARC-G, P05-G, ARC-C and P05-C, which reproduces its cells of Table III at 1M and 21M.

as few pages as HYPERPAGE at 1% of the 1M index, and 1.5 to 1.75 times at 1% of the full corpus. The factor shrinks as the budget grows. At 1% of the 21M index, the extra 68 to 103 MiB are 5% to 8% of the engine's peak resident memory, so at this scale the layout saves page reads more than memory. Charged the measured memory of DiskANN's node cache instead of the declared share, HYPERPAGE still saves 18.20% to 29.02% at 1M on every draw.

The budget must enter refinement: Logged layouts that ignore the cache lose to MARGO at the large budgets of the full corpus, because they spend their gain on cached records. Under a pure page LRU, our four cache-blind objectives beat MARGO at all seven budgets, by 14.22% to 40.02% ($\rho = 0$ in Fig. 3, which shows three budgets). At 25% of the full corpus, they read more than MARGO for every share from 0.5 to 0.9. At 15% and 25%, they read 3.5% to 11.5% more at their validation shares. These shares cache 11.2% and 22.5% of the records, which serve 74.8% and 84.7% of the test accesses and carry 79% and 92% of the training incidences. By Corollary 3, these records add nothing to the reads of any scope, and leaving them out of refinement reverses the loss (Section VII-C). A cache-aware layout is tied to its budget: at the other large budget it reads 0.8% to 3.0% more than MARGO on validation steps, so HYPERPAGE keeps the incumbent among the candidates. A cache-aware layout also needs a record cache. If the share holds the most accessed pages instead, P05-C's saving falls to 8.43%, -3.33% and 1.56% at 1M.

Why the baselines fall short: Orders without logs gain little, and no log-aware baseline reads fewer pages than HYPERPAGE in any column. STARLING saves at most 2.8% at 1M and 3.6% to 5.3% on the full corpus, second only to HYPERPAGE at 25%. Proximity and RCM read more pages than MARGO in all but one cell. HEAT, deployed cache-aware on every draw, saves 5.4 to 7.8 points less than HYPERPAGE on W3-1M, and 3.0 and 4.0 points less at 1% and 5% of the full corpus. Our residual unions raise both Mt-KaHyPar and HEAT in 9 of 10 columns, the exception being W4 at 5%. Mt-KaHyPar on the same residual unions saves 10.7 to 12.1 points less on W3-1M. On the full corpus it reads 2.2% to 7.8% more than MARGO, although it lowers the connectivity of the training unions by 18% to 26%. P05-C's layouts also have 11.7% to 12.0% lower connectivity of the residual training unions than the two-level partitioner's, its own objective, so the gap lies in optimization at exact capacity. Training connectivity is thus an unreliable guide to held-out reads. Under one shared cache, residual training connectivity ranks the validation reads of each W3 column's 18 layouts with Spearman's r_s of 0.06 to 0.92. Validation replay ranks test reads at 0.92 to 1.00, so HYPERPAGE selects by replay.

Scale: The saving persists as the index grows (Fig. 4, with uniform 5M and 10M samples of the corpus and the same queries). At every size from 1M to 21M, HYPERPAGE reads 12.5% to 21.2% fewer pages than MARGO at 1% and 5% and saves more than either cache-blind objective. At 1%, its saving falls by 6.1 points from 1M to 5M and by 2.6 more to the full corpus, consistent with the larger share of accesses that 1% of a larger index serves (Section I). The leading objective also changes, from P05-G at 1M to ARC-G on the full corpus, where the training unions cover 33% of the records instead of 86%.

Stability over the run: A layout learned once keeps its advantage across the evaluation run. From step 1 to step 240, the savings of the frozen 1M deployments fall by only 2.0 to 2.6 points. On full-corpus steps 1 to 40 and 201 to 240, which no selection used, the frozen full-corpus deployments read 4.7% to 22.1% and 4.9% to 12.5% fewer pages than MARGO. Each also beats every other candidate at its validation share on every draw. We fix the graph, so inserts and deletes stay outside this study.

Search time: HYPERPAGE's fewer page reads cut the search time in the hybrid-cache engine at every budget we time (Table IV). The median paired saving is 9.60% to 13.87% per step at 1M and 2.59% to 10.38% on the full corpus. The table times MARGO and HYPERPAGE's timed layout of each budget over the 130 test steps, in alternating pairs. A pair is void when a check fixed before the run flags host load, CPU affinity or the read speed of the two files. Two pairs were void, one at 25% of 1M and one at 1% of the full corpus. At every budget, HYPERPAGE is faster in every valid pair, and each budget with 10 pairs gives a sign-test p of 0.002. The time saving is smaller than the read saving because the engine reads the pages of one beam round in parallel, so time follows the rounds that touch the disk. Median sub-query latency falls at every budget, by 2.2% to 16.1%, and

TABLE IV

SEARCH TIME PER W3 TEST STEP IN THE HYBRID-CACHE ENGINE (12 THREADS, ONE HOST). TIME SAVED IS THE MEDIAN OF THE PAIRED SAVINGS. THE 1M ROWS AT 1% AND 5% TIME LAYOUTS SELECTED IN THE ENGINE'S ARRIVAL ORDER, NOT THE DEPLOYMENTS OF TABLE III. ROWS 21M 1% AND 5% COME FROM ROUNDS OF FIVE ARMS, SO THEIR TWO ARMS ARE NOT ADJACENT.

			s per step		reads	time
Budget	layout	pairs	MARGO	arm	saved%	saved%
1M, 1%	CLQ-G	10	5.74	5.00	18.82	12.85
1M, 5%	P05-C	10	3.73	3.21	20.38	13.87
1M, 25%	ARC-C	10	1.79	1.62	16.60	9.60
21M, 1%	ARC-C	3	6.58	6.06	12.65	7.80
21M, 5%	ARC-C	3	3.98	3.57	17.04	10.38
21M, 15%	HEAT, res.	10	2.11	2.06	5.28	2.59
21M, 25%	MARGO, uncached	10	1.77	1.71	6.72	3.48

TABLE V

MARGO'S PAGE-SEARCH ENGINE AGAINST THE HYBRID CACHE (12 THREADS, THE HOST OF TABLE IV): PAGE READS PER TEST CALL / SECONDS PER TEST STEP, MEDIANS OF 3, 4 AND 2 VALID ROUNDS AT 1M AND 3 AT 21M. PAGE SEARCH TAKES THE SMALLEST L WITH VALIDATION RECALL@10 OF 0.95. TEST RECALL@10 IS 0.946 TO 0.954. ITS 1M HYBRID ARMS AT 1% AND 5% ARE THOSE OF TABLE IV. AT 25% IT TIMES P05-C ON OLDER COPIES OF THE FILES, WHERE MARGO TAKES 1.85 S PER STEP INSTEAD OF 1.79 S.

	Page search		Hybrid cache	
Budget	MARGO	HYPERPAGE	MARGO	HYPERPAGE
1M, 1%	200.3 / 6.08	176.5 / 5.44	331.3 / 5.74	269.0 / 5.00
1M, 5%	149.6 / 4.83	137.0 / 4.53	175.0 / 3.73	139.4 / 3.21
1M, 25%	78.4 / 3.08	72.4 / 3.02	60.7 / 1.85	51.1 / 1.67
21M, 1%	296.9 / 9.66	268.9 / 8.88	319.7 / 6.58	279.3 / 6.06
21M, 5%	187.2 / 6.89	208.4 / 7.82	155.4 / 3.98	128.9 / 3.57

p99 falls by 2.1% to 6.3%. At 1% and 5% of 1M under the lockstep shares, 12 concurrent sub-queries share a 417-page LRU, and the engine reads 31% to 38% more than the replay. Replayed in the arrival order that the engine logs with MARGO's layout, MARGO's reads match the engine's within 0.001%. The timed 1M layouts at 1% and 5% are selected under this order, which approximates each candidate's own order. They were selected before STARLING, ARC-CF and the HEAT and MARGO layouts on uncached records joined the candidates. On the full corpus, where the LRU holds thousands of pages, the engine reads 0.64% and 0.38% fewer pages than the replay at 1% and 5%. Both arms of every pair reach the same peak resident memory within 0.4%, PQ codes and engine buffers included. The public logs carry no step timing, so we cannot report retrieval's share of a training step, and these times measure the retrieval tier alone. Payback charges the refinement in core-seconds and the index rewrite. With 400 steps per run, a layout refined once pays for itself within 0.8 to 3.1 later runs in core-seconds at 1M and at 1% and 5% of the full corpus. In 12-thread wall time it needs 9.7 to 37.5 runs there. At 15% and 25% of the full corpus, where it saves 2.59% and 3.48% of the time, it needs 45.9 and 6.6 runs in core-seconds and 551 and 79 runs of wall time.

MARGO's page-search engine: The hybrid engine with HYPERPAGE's layout is the fastest configuration we time, and HYPERPAGE also speeds up MARGO's own engine at 1M (Table V). MARGO's engine scores every record of a fetched page, so its walks depend on the layout and fall outside A1. We port it to 8 KB pages with a record cache of the whole budget ranked by training frequency, which beats its own cache selection, and refine HYPERPAGE on the records that this cache leaves uncached. At 1M, HYPERPAGE's page-search layouts read 11.8%, 8.3% and 7.5% fewer pages than MARGO's on every draw, at test recall within 0.3 points. With either layout, page search takes 1.06 to 2.19 times as long as the hybrid engine and holds 20% to 33% more memory. At 1M, HYPERPAGE's layouts take 9.8% to 14.0% less time than MARGO's in the hybrid engine and 1.9% to 10.6% less in page search, in every round. On the full corpus, its page-search layout takes 8.2% less time at 1%. At 5% it needs $L = 700$ instead of 600 to reach the recall target, so it reads 11.4% more pages and takes 13.6% more time. In the hybrid engine, HYPERPAGE takes 1.44% and 2.62% less time than HEAT on residual unions, the fifth arm on the full corpus. At 5% with 64 to 256 concurrent sub-queries, it serves about twice the sub-queries per second of page search with MARGO's layout, at lower p99 latency. MARGO's hybrid runs slowed within each run at this load, so we leave them out.

Transfer to another agent: Layouts learned from one agent's run also serve another agent, but the cache split must be chosen anew. The W4 columns of Table III replay the held-out traversals of an OLMo-3-7B agent at eight checkpoints on the same 1M index (test recall@10 0.874). Every layout and hot-record ranking comes from the W3 training run, and only the share and the deployed layout are chosen on W4 validation steps. HYPERPAGE reads 19.4%, 14.0% and 14.5% fewer pages than MARGO at 1%, 5% and 25% on every draw, and its saving moves by at most 1.21 points from the first checkpoint to the last. At 5%, MARGO's share moves from 0.90 on W3 to 0.75 on W4, so selection deploys P05-G instead of the layouts refined for W3's hot set.

Read-once scopes: Under read-once buffers, every objective of HYPERPAGE beats MARGO at both scales, and only Mt-KaHyPar comes close, at 1M (Table VI). Under the keyed group buffer, our objectives read 14.99% to 20.81% fewer pages than MARGO at 1M and 9.64% to 13.65% fewer on the full corpus. Mt-KaHyPar saves 12.44% at 1M but 0.63% on the full corpus. A keyed buffer needs memory, however: the 128 live groups of a step hold 144,703 pages at 1M, more than the 83,334-page index, and 2.54 to 2.95 GiB on the full corpus.

C. Ablation Study

Both steps of HYPERPAGE matter, and the budget decides which one carries the saving (Table VII). Learning from logs alone, the better cache-blind layout saves 8.2% to 20.0% at 1M and at 1% and 5% of the full corpus. Refining ARC cache-aware adds 1.2 to 11.9 points in every W3 column and turns the losses at 15% and 25% of the full corpus into savings. Even

TABLE VI
READS UNDER READ-ONCE SCOPES ON W3 TEST: MEAN READS PER GROUP AND THE SAVING Δ OVER MARGO (%) PER GROUP AND PER TOOL CALL. BOLD MARKS THE BEST AND UNDERLINE THE SECOND BEST PER COLUMN.

	W3-1M			W3-21M		
Layout	group↓	Δ	call Δ	group↓	Δ	call Δ
STARLING	1,408.1	−0.22	1.32	2,944.6	2.40	3.01
MARGO	1,405.0	0.00	0.00	3,017.0	0.00	0.00
FREQ	1,622.5	−15.47	−17.76	3,439.1	−13.99	−17.31
Mt-KaHyPar	1,230.2	12.44	10.58	2,998.1	0.63	−0.46
SHP-G	1,404.0	0.08	0.09	3,019.2	−0.07	−0.09
ARC-G	1,194.4	14.99	17.08	**2,605.1**	**13.65**	**15.02**
CLQ-G	1,147.8	18.31	<u>20.53</u>	2,726.1	9.64	11.20
SET-G	<u>1,130.5</u>	<u>19.54</u>	19.76	2,659.8	11.84	11.70
P05-G	**1,112.7**	**20.81**	**21.77**	<u>2,644.5</u>	<u>12.35</u>	<u>13.07</u>

TABLE VII
WHERE HYPERPAGE'S SAVING COMES FROM: SAVING OVER MARGO (%) OF HYPERPAGE'S CANDIDATES. BOLD MARKS HYPERPAGE'S PICK ON VALIDATION.

	W3-1M			W3-21M			
Layout	1%	5%	25%	1%	5%	15%	25%
Logs, cache-blind							
ARC-G	16.3	12.1	4.7	11.3	14.1	−3.5	−7.6
P05-G	20.0	14.1	8.2	5.8	10.3	−10.0	−10.0
Plus cache-aware refinement							
ARC-C	18.1	15.6	**16.6**	**12.5**	**16.9**	3.1	1.2
P05-C	**21.2**	**16.4**	15.8	7.3	10.1	−5.7	−3.5
ARC-CF	17.0	14.6	15.2	8.1	9.7	2.4	1.9
HEAT's objective	13.4	10.3	11.2	9.6	13.0	**5.2**	2.6
MARGO's objective, cache-aware							
MARGO on uncached records	0.3	1.3	5.9	0.3	−0.9	4.6	**6.8**

without selection, this fixed recipe, ARC-C, beats MARGO in every W3 column, by 1.2% to 18.1%. At these two budgets, most of the gain comes from leaving cached records out rather than from the logged objective. MARGO on uncached records partitions the graph induced by those records with MARGO's code and fills the free slots with the cached ones. It saves 4.6% and 6.8% there, while the best logged objective adds 0.6 points at 15% and loses 4.2 at 25%. Elsewhere it saves at most 5.9%, and at 5% of the full corpus it reads 0.9% more than MARGO, a margin not significant with moving blocks. No layout is best in every column, so HYPERPAGE selects on validation, and in every column its pick is also the best of its candidates on test.

Within the cache-blind step, logging at all gives most of the saving, and each refinement of the objective adds a little. Under read-once scopes at 1M, an arc-restricted objective on logged group unions saves 14.99% against MARGO. Dropping the arc restriction adds 3.90%, as the off-arc share of co-located pairs rises from 0.404 to 0.787. The set form and P05 add 1.51% and 1.58% more with the same start, candidates and passes, the first consistent with Propositions 1 and 2. The move rule matters too: on the P05 objective from the same start, SHP's move rule saves 0.08% where P05-G saves 20.81%. Of the gain that SHP predicts on a round's snapshot, 78% to 80% does not materialize.

The logs mostly supply the query distribution and the co-access inside sub-queries. Refining on sub-queries shuffled across the groups of a step keeps 16.28% of the 19.54% at group scope. An exact-string result cache of at most 1/64 of the budget complements the layout: it saves MARGO 16.82% to 25.17% of its reads and leaves HYPERPAGE 12.14% to 20.59% ahead.

Other workloads and page size: The saving holds on other agent workloads and smaller pages, and it depends on how much the queries of a batch overlap. On W1 and W2 under the hybrid cache, the candidates are MARGO, the four cache-blind layouts and P05-C, and MARGO's share is chosen per draw. HYPERPAGE then reads 16.71% to 24.24% fewer test pages than MARGO on W1 and 20.50% to 25.38% fewer on W2. On DEEP1M, whose random batches overlap by 7.6%, the cache-blind layouts save only 0.33% to 4.74%, against 18.62% for ARC-G at 1% on W1. With 2 to 6 records per page and candidates refined anew, HYPERPAGE still reads less than MARGO on W3-1M on every draw. At 1% and 5%, its saving falls with B, since each record has at most $B-1$ page partners: it saves 13.96% and 12.00% with 6 records per page and 4.49% and 5.56% with 2.

D. Optimization Audit

The learned layouts read at most 2.05 to 2.20 times the optimum under read-once group scopes, against 2.59 times for MARGO, but the bound leaves the true gap open. On the cold read-once group scopes of Table VI, Proposition 7 with 67 record classes gives 543.2 reads per W3-1M test group, 2.81 times the capacity bound L_{cap}. The best layout we found, refined on the test groups themselves and so only an in-sample reference, reads 974.4, and longer runs changed it by under 0.5%. Hence $543.2 \le R^{\star} \le 974.4$, which gives the ratios above. We also solve 71 smaller W3 instances to proven optimality, with 12 to 48 records and B from 2 to 12. On them, the bound with one class per record reaches a median of 96% of the optimum at $B=3$ and 81% to 90% at $B=12$. The bound is thus nearly tight at small B, and its looseness at $B=12$ leaves open how much a better layout could still save. Under the shared cache of Table III, the window bound of Proposition 8 lies 11 to 48 times below HYPERPAGE's reads, so the optimum of the primary metric remains open.

VIII. CONCLUSION

The logs of one finished agent run price every page layout of a fixed disk-resident graph index, and cached records add no page reads. Choosing a layout for a declared budget thus becomes a measurable optimization problem. HYPERPAGE builds layouts on the uncached part of the logged traversals and selects one by replay. On the main workload it reads up to 21.2% fewer pages than MARGO, most when the cache is small. Once the cache serves most accesses, the budget must enter refinement. A Lagrangian bound brackets the remaining gap to the optimum. Index updates, larger corpora and the optimum under a shared cache remain open.

REFERENCES

[1] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.

[2] S. Jayaram Subramanya, Devvrit, R. Kadekodi, R. Krishnaswamy, and H. V. Simhadri, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[3] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," arXiv:2402.03300, 2024.

[4] B. Jin, H. Zeng, Z. Yue, J. Yoon, S. Arik, D. Wang, H. Zamani, and J. Han, "Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning," arXiv:2503.09516, 2025.

[5] M. Wang, W. Xu, X. Yi, S. Wu, Z. Peng, X. Ke, Y. Gao, X. Xu, R. Guo, and C. Xie, "Starling: An I/O-efficient disk-resident graph index framework for high-dimensional vector similarity search on data segment," *Proceedings of the ACM on Management of Data*, vol. 2, no. 1, pp. 1–27, 2024.

[6] Z. Yue, B. Zheng, L. Xu, K. Xu, S. Zhang, Y. Du, Y. Gao, X. Zhou, and C. S. Jensen, "Select edges wisely: Monotonic path aware graph layout optimization for disk-based ANN search," *Proceedings of the VLDB Endowment*, vol. 18, no. 11, pp. 4337–4349, 2025.

[7] A. Eisenman, M. Naumov, D. Gardner, M. Smelyanskiy, S. Pupyrev, K. Hazelwood, A. Cidon, and S. Katti, "Bandana: Using non-volatile memory for storing deep learning models," in *Proceedings of Machine Learning and Systems (MLSys)*, 2019, arXiv:1811.05922.

[8] I. Kabiljo, B. Karrer, M. Pundir, S. Pupyrev, and A. Shalita, "Social hash partitioner: A scalable distributed hypergraph partitioner," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1418–1429, 2017.

[9] S. Jaiswal, R. Krishnaswamy, A. Garg, H. V. Simhadri, and S. Agrawal, "OOD-DiskANN: Efficient and scalable graph ANNS for out-of-distribution queries," arXiv:2211.12850, 2022.

[10] J. Ni, X. Xu, Y. Wang, C. Li, J. Yao, S. Xiao, and X. Zhang, "DiskANN++: Efficient page-based search over isomorphic mapped graph index using query-sensitivity entry vertex," arXiv:2310.00402, 2023.

[11] D. Kang, D. Jiang, H. Yang, H. Liu, and B. Li, "Scalable disk-based approximate nearest neighbor search with page-aligned graph," arXiv:2509.25487v2, 2025.

[12] P. Yin, X. Yan, Q. Zhou, H. Li, X. Li, M. Wang, L. Zhang, X. Yao, and J. Cheng, "GPS: Revisiting the data layout for disk-based high-dimensional vector search," *Proceedings of the ACM on Management of Data*, vol. 4, no. 3, pp. 1–29, 2026.

[13] H. Li, X. Huang, B. Choi, and J. Xu, "BAMG: A block-aware monotonic graph index for disk-based approximate nearest neighbor search," in *Proceedings of the IEEE 42nd International Conference on Data Engineering (ICDE)*, 2026, pp. 3433–3445.

[14] W. Zhao, Y. Lu, Y. Tian, H. Zhang, J. Li, M. Zhao, Y. Li, and W. Qian, "Optimizing SSD-resident graph indexing for high-throughput vector search," arXiv:2602.22805, 2026.

[15] J. Lou, S. Gong, Q. Yu, H. Guo, Y. Lu, S. Yu, Y. Zhang, T. Nie, and G. Yu, "DGAI: Decoupled on-disk graph-based ANN index for efficient updates and queries," arXiv:2510.25401v5, 2025.

[16] Y. Zhou, S. Lin, S. Gong, S. Yu, S. Fan, Y. Zhang, and G. Yu, "GoVector: An I/O-efficient caching strategy for high-dimensional vector nearest neighbor search," arXiv:2508.15694, 2025.

[17] S. Zhong, D. Mo, and S. Luo, "LSM-VEC: A large-scale disk-based system for dynamic vector search," arXiv:2505.17152v2, 2025.

[18] Q. Liang, Z. Chen, Y. Yu, and N. Xiao, "CoWalk: Cooperative walks for agentic batch queries on disk-resident graphs," arXiv:2610.38404, 2026.

[19] J. Coelho, J. Ning, J. He, K. Mao, A. Paladugu, P. Setlur, J. Jin, J. Callan, J. Magalhães, B. Martins, and C. Xiong, "DeepResearchGym: A free, transparent, and reproducible evaluation sandbox for deep research," arXiv:2505.19253, 2025.

[20] G. Zhang, J. Zhu, R. Yang, K. Qiu, M. Zhang, Z. Wu, Q. Dai, B. Liu, C. Luo, Z. Yang, L. Li, L. Wang, W. Chen, Y. Zhang, X. Li, Z. Liu, X. Geng, and B. Guo, "InfoAgent: Advancing autonomous information-seeking agents," arXiv:2509.25189, 2025.

[21] P. Hua and Y. Xiao, "Semantics delivery network: Rethinking web retrieval infrastructure for LLM agents," in *Proceedings of the 25th ACM Workshop on Hot Topics in Networks (HotNets)*, 2026, to appear; arXiv:2609.22486.

[22] G. Karypis, R. Aggarwal, V. Kumar, and S. Shekhar, "Multilevel hypergraph partitioning: Applications in VLSI domain," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 7, no. 1, pp. 69–79, 1999.

[23] L. Gottesbüren, T. Heuer, N. Maas, P. Sanders, and S. Schlag, "Scalable high-quality hypergraph partitioning," *ACM Transactions on Algorithms*, vol. 20, no. 1, pp. 1–54, 2024.

[24] R. Fan, M. Xie, H. Jiang, and Y. Lu, "MaxEmbed: Maximizing SSD bandwidth utilization for huge embedding models serving," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 4*, 2024, pp. 188–202.

[25] C. Curino, E. Jones, Y. Zhang, and S. Madden, "Schism: A workload-driven approach to database replication and partitioning," *Proceedings of the VLDB Endowment*, vol. 3, no. 1–2, pp. 48–57, 2010.

[26] Z. Yang, B. Chandramouli, C. Wang, J. Gehrke, Y. Li, U. F. Minhas, P.-Å. Larson, D. Kossmann, and R. Acharya, "Qd-tree: Learning data layouts for big data analytics," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2020, pp. 193–208.

[27] Z. Li, M. L. Yiu, and T. N. Chan, "PAW: Data partitioning meets workload variance," in *Proceedings of the IEEE 38th International Conference on Data Engineering (ICDE)*, 2022, pp. 123–135.

[28] M. M. Tsangaris and J. F. Naughton, "A stochastic approach for clustering in object bases," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1991, pp. 12–21.

[29] ——, "On the performance of object clustering techniques," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1992, pp. 144–153.

[30] C. Gerlhof, A. Kemper, C. Kilger, and G. Moerkotte, "Partition-based clustering in object bases: From theory to practice," in *Foundations of Data Organization and Algorithms (FODO)*, ser. Lecture Notes in Computer Science. Springer, 1993, pp. 301–316.

[31] M. H. Nodine, M. T. Goodrich, and J. S. Vitter, "Blocking for external graph searching," *Algorithmica*, vol. 16, no. 2, pp. 181–214, 1996.

[32] A. Yaşar, B. Gedik, and H. Ferhatosmanoğlu, "Distributed block formation and layout for disk-based management of large-scale graphs," *Distributed and Parallel Databases*, vol. 35, no. 1, pp. 23–53, 2017.

[33] R. Liu, Y. Wang, X. Yan, H. Jiang, Z. Cai, M. Wang, B. Tang, and J. Li, "DiskGNN: Bridging I/O efficiency and model accuracy for out-of-core GNN training," *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, pp. 1–27, 2025.

[34] Y. Lim, U. Kang, and C. Faloutsos, "SlashBurn: Graph compression and mining beyond caveman communities," *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 12, pp. 3077–3089, 2014.

[35] H. Wei, J. X. Yu, C. Lu, and X. Lin, "Speedup graph processing by graph ordering," in *Proceedings of the International Conference on Management of Data (SIGMOD)*, 2016, pp. 1813–1828.

[36] B. Coleman, S. Segarra, A. Smola, and A. Shrivastava, "Graph reordering for cache-efficient near neighbor search," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[37] Z. Zhuang, N. Xiao, Z. Chen, and Y. Yu, "ConStore: Search-overlapped prefetch and co-retrieval-aware content storage for disk-based ANNS," arXiv:2610.21676, 2026.

[38] J. Mohoney, A. Pacaci, S. R. Chowdhury, A. Mousavi, I. F. Ilyas, U. F. Minhas, J. Pound, and T. Rekatsinas, "High-throughput vector similarity search in knowledge graphs," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–25, 2023.

[39] J. Mohoney, D. Sarda, M. Tang, S. R. Chowdhury, A. Pacaci, I. F. Ilyas, T. Rekatsinas, and S. Venkataraman, "Quake: Adaptive indexing for vector search," in *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2025.

[40] W. Wei, D. Lian, Q. Feng, and Y. Wu, "Robust tree-based learned vector index with query-aware repartitioning," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025, pp. 3134–3143.

[41] Z. Zhang and W. Zheng, "Beyond structure-driven tuning: Cost-aligned graph optimization for approximate nearest neighbor search," in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2026, pp. 6700–6711.

[42] S. Abuzakuk, A.-M. Kermarrec, R. Pires, M. Randl, and M. de Vos, "Catapults to the rescue: Accelerating vector search by exploiting query locality," arXiv:2603.02164, 2026.

[43] R. Wan, X. Zeng, Y. Qiao, and H. Zhang, "WaRP: Workload-aware request planning for vector retrieval on object storage," arXiv:2610.29689, 2026.

[44] L. Li, Z. He, Z. Zhang, and Y. Deng, "ANN-Cache: Accelerating approximate nearest neighbor search via caching," *IEEE Transactions on Knowledge and Data Engineering*, vol. 38, no. 9, pp. 5636–5649, 2026.

[45] J. Shim, J. Oh, H. Roh, J. Do, and S.-W. Lee, "Turbocharging vector databases using modern SSDs," *Proceedings of the VLDB Endowment*, vol. 18, no. 11, pp. 4710–4722, 2025.

[46] A. Al-Mamun, "Cross-modal data co-locality for heterogeneous storage systems," arXiv:2610.34595, 2026.

[47] M. Chen, K. Zhang, Z. He, Y. Jing, and X. S. Wang, "RoarGraph: A projected bipartite graph for efficient cross-modal approximate nearest neighbor search," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 2735–2749, 2024.

[48] S. Chaudhuri and V. Narasayya, "AutoAdmin "what-if" index analysis utility," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1998, pp. 367–378.

[49] H. Guo and Y. Lu, "Achieving low-latency graph-based vector search via aligning best-first search algorithm with SSD," in *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2025, pp. 171–186.

[50] L. Li, S. Gong, Y. Yang, Y. Wang, and J. Wu, "I/O optimizations for graph-based disk-resident approximate nearest neighbor search: A design space exploration," *Proceedings of the VLDB Endowment*, vol. 19, no. 7, pp. 1484–1498, 2026.

[51] J. Edmonds, "Maximum matching and a polyhedron with 0,1-vertices," *Journal of Research of the National Bureau of Standards, Section B*, vol. 69B, no. 1–2, pp. 125–130, 1965.

[52] D. G. Kirkpatrick and P. Hell, "On the completeness of a generalized matching problem," in *Proceedings of the 10th Annual ACM Symposium on Theory of Computing (STOC)*, 1978, pp. 240–245.

[53] M. Chlebík and J. Chlebíková, "Complexity of approximating bounded variants of optimization problems," *Theoretical Computer Science*, vol. 354, no. 3, pp. 320–338, 2006.

[54] P. Alimonti and V. Kann, "Some APX-completeness results for cubic graphs," *Theoretical Computer Science*, vol. 237, no. 1–2, pp. 123–134, 2000.

[55] M. L. Fisher, "The Lagrangian relaxation method for solving integer programming problems," *Management Science*, vol. 27, no. 1, pp. 1–18, 1981.

[56] I. Elhallaoui, D. Villeneuve, F. Soumis, and G. Desaulniers, "Dynamic aggregation of set-partitioning constraints in column generation," *Operations Research*, vol. 53, no. 4, pp. 632–645, 2005.

[57] P. Erdős and H. Sachs, "Reguläre Graphen gegebener Taillenweite mit minimaler Knotenzahl," *Wissenschaftliche Zeitschrift der Martin-Luther-Universität Halle-Wittenberg, Mathematisch-Naturwissenschaftliche Reihe*, vol. 12, pp. 251–258, 1963.

[58] C. M. Fiduccia and R. M. Mattheyses, "A linear-time heuristic for improving network partitions," in *Proceedings of the 19th Design Automation Conference (DAC)*, 1982, pp. 175–181.

[59] E. Cuthill and J. McKee, "Reducing the bandwidth of sparse symmetric matrices," in *Proceedings of the 24th National Conference of the ACM*, 1969, pp. 157–172.

[60] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[61] Z. Li, D. Jiang, X. Ma, H. Zhang, P. Nie, Y. Zhang, K. Zou, J. Xie, Y. Zhang, and W. Chen, "OpenResearcher: A fully open pipeline for long-horizon deep research trajectory synthesis," arXiv:2603.20278, 2026.

[62] A. Babenko and V. Lempitsky, "Efficient indexing of billion-scale datasets of deep descriptors," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2055–2063.

[63] H. V. Simhadri, G. Williams, M. Aumüller, M. Douze, A. Babenko, D. Baranchuk, Q. Chen, L. Hosseini, R. Krishnaswamy, G. Srinivasa, S. Jayaram Subramanya, and J. Wang, "Results of the NeurIPS'21 challenge on billion-scale approximate nearest neighbor search," in *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*, ser. Proceedings of Machine Learning Research, vol. 176, 2022, pp. 177–189.

[64] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2024, pp. 641–649.